EXPLAINER · INTERVIEW PREP

nanochat-d20, explained

A 20-layer chat model trained from scratch on one H100. This page explains every stage and every term in the README, with the actual numbers from the run.

The 30-second version

897M	3.48B	~6 h	0.236
parameters (393M in the transformer)	pretraining tokens	end-to-end on 1× H100	CORE (GPT-2 355M: 0.192)

Everything was learned from random initialization: the tokenizer, the language model and the chat behaviour. The code is nanochat (Andrej Karpathy's minimal LLM training stack); the work here is choosing the configuration, running it reliably on rented hardware, evaluating it fairly and shipping it.

The pipeline (click a stage)

1. Tokenizer ~2 min | **2. Pretrain** ~5.2 h | **3. Chat SFT** 47 min | **4. RL** 42 min | **5. Evaluate** ~30 min | **6. Serve** on demand

PRETRAIN · ~5.2 H

Next-token prediction on 3.48B tokens: 3,320 steps of 1M tokens, Muon + AdamW, FP8. Validation loss falls from 3.17 to 0.755 bits/byte.

Tokenizer	~2 min	Train a 32,768-token BPE vocabulary on ~2B characters of the pretraining data. Turns text into integer ids.
Pretrain	~5.2 h	Next-token prediction on 3.48B tokens: 3,320 steps of 1M tokens, Muon + AdamW, FP8. Validation loss falls from 3.17 to 0.755 bits/byte.
Chat SFT	47 min	467 steps on ~790k conversations (SmolTalk + MMLU + GSM8K). Learns turn-taking and answer formats. Loss only on assistant tokens.
RL	42 min	75 steps: sample 16 answers per GSM8K problem, reward correct final numbers, reinforce better-than-average answers. GSM8K 1.5% → 10.0%.
Evaluate	~30 min	CORE (22 tasks) on the base model; ARC, MMLU, GSM8K and HumanEval on the chat model; GPT-2 scored with the same harness.
Serve	on demand	Gradio UI + OpenAI-compatible API. Free CPU Space for the public demo; small on-demand GPU behind a fixed HTTPS relay.

1 · Tokenizer

A model reads integers, not text. The tokenizer maps text to a sequence of **token ids** drawn from a fixed vocabulary. We trained our own **BPE** (byte-pair encoding) tokenizer with a vocabulary of **32,768** on ~2B characters of the pretraining data.

> **How BPE works.** Start from raw bytes. Repeatedly find the most frequent adjacent pair and merge it into a new token. Common words end up as one token (`sunlight`); rare words are split into pieces (`K|erala`). Leading spaces are part of tokens, which is why most tokens below start with a space.

The	monsoon	arrives	in	K	erala	in	early	June	.
483	24356	17710	287	776	31330	287	1801	4839	46

44 characters → 10 tokens (4.4 chars/token)

Photos	ynthesis	converts	sunlight	into	chemical	energy	.
23144	8116	14817	4670	609	2019	1063	46

def	add	(a	,	b	):↵		return	a	+	b
13210	729	14064	44	277	12383	6091	3291	257	3122	277

To	ken	ization	splits	unbelie	v	ably	long	words	into	pieces	.
2058	3594	1481	27051	30398	118	1621	818	2791	609	3464	46

These splits come from the actual trained tokenizer. A typical English token is about 4–5 characters, which is why "bits per byte" (below) is used to compare models with different tokenizers.

2 · Architecture

A decoder-only Transformer, the same family as GPT-2/Llama, with modern refinements. The single knob is **depth = 20**; nanochat derives everything else from it: width = 20 × 64 = **1280**, 10 attention heads of 128 dimensions, context of **2,048** tokens.

The 20 layers (click one)

S: sliding window, sees the last 768 tokens L: full attention, sees all 2,048 has a value embedding



Layer 3 · full attention over all 2,048 tokens · **value embedding**: a per-token learned vector (looked up by token id) is added into the attention values through a learned gate, giving this layer direct access to token identity. Each layer: attention (6.6M weights) + MLP (13.1M weights).

Inside one block

input from the previous layer (1280 numbers per token)

↓

RMSNorm (no learned scale)

↓

Self-attention: Q, K, V projections → **RoPE** positions → **QK-norm** → FlashAttention-3 with this layer's window → output projection

+ residual: add the input back

↓

RMSNorm

↓

MLP: 1280 → 5120, **relu²**, 5120 → 1280

+ residual

Attention lets each token look back at earlier tokens and pull in information. **The MLP** then processes each token on its own; it holds much of the model's "knowledge".

Residual connections mean each block adds a correction instead of replacing its input, so very deep stacks still train stably.

Model-level extras nanochat adds around the stack:

- **Smear**: mixes a gated bit of the previous token's embedding into each token (cheap bigram signal).
- **x0 / residual lambdas**: learned per-layer scalars that blend the original embedding back in.
- **Backout**: subtracts a fraction of the layer-10 activations before the output, removing low-level features.
- **Logit soft-cap**: output scores pass through 15·tanh(x/15), which prevents extreme values and stabilizes training.

Where the 897M parameters live



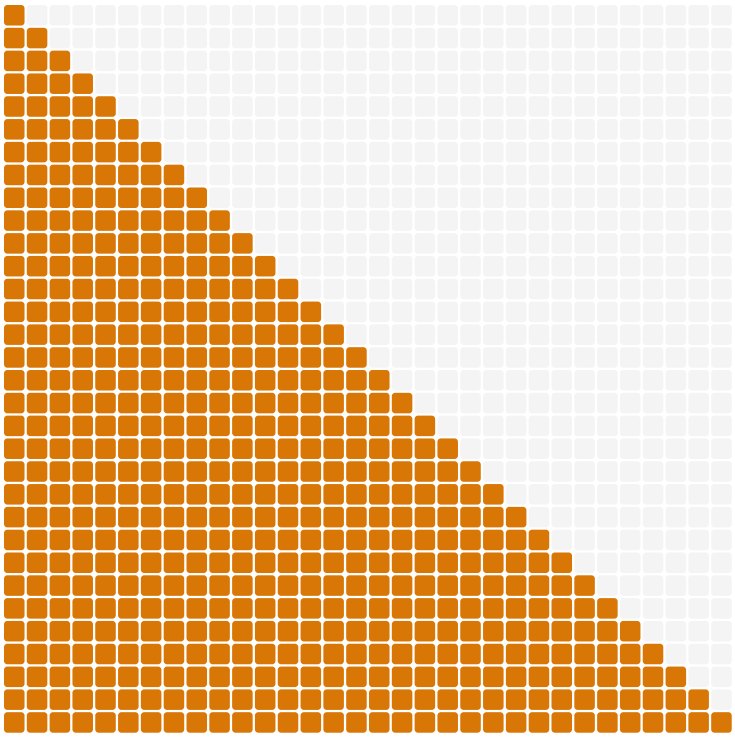
- **262M** MLP weights (20 × 13.1M)
- **131M** Attention weights (20 × 6.6M)
- **419M** Value embeddings (10 layers × 32,768 × 1280)
- **42M** Token embedding (32,768 × 1280)
- **42M** Output layer (1280 × 32,768)

Transformer (compute) = 393M · lookup tables = 503M

> Why two numbers (897M vs 393M)? **Embedding tables** are lookups: a token id selects one row, so they cost memory but almost no compute. The **393M transformer weights** do the multiplications on every token, which is the fairer number when comparing compute and capability (GPT-2's counts include its much smaller embeddings).

Sliding-window vs full attention

Each square is "token *row* may look at token *column*". Causal models only look backwards (lower triangle). A sliding window limits how far back. Here 32 squares stand for 2,048 tokens; the window is a quarter.

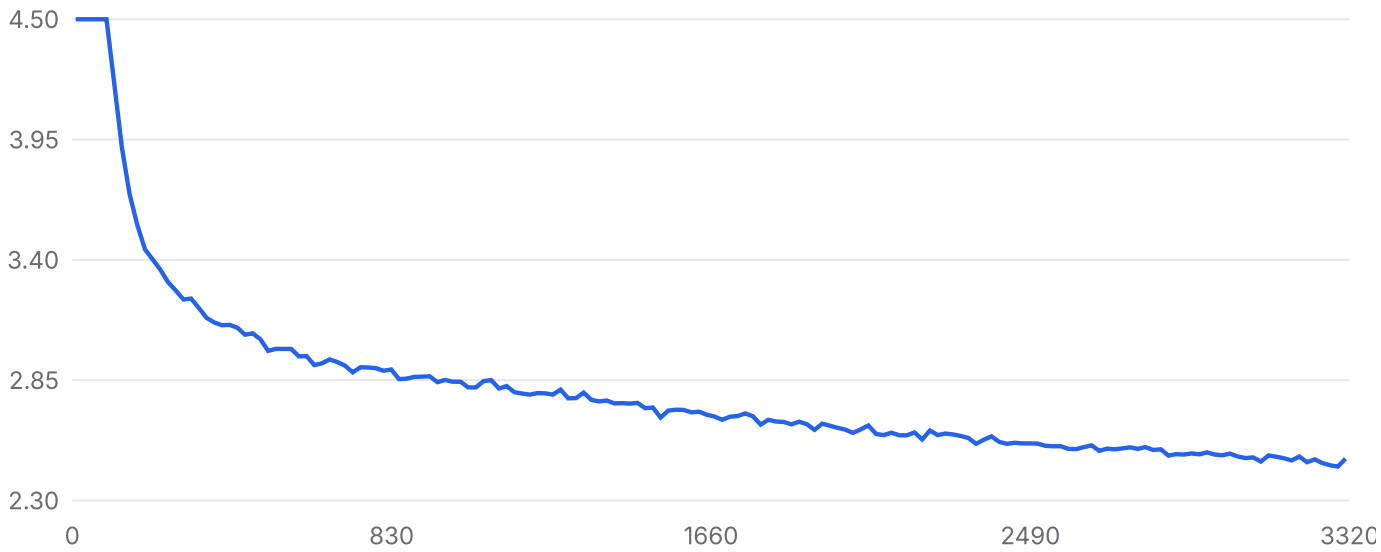


Why mix them? Sliding windows make 15 of 20 layers cheaper (attention cost grows with the square of what each token sees), while the 5 full layers still carry long-range information across the whole 2,048-token context.

3 · Pretraining

The model reads web text and learns one thing: **predict the next token**. Doing this well across billions of tokens forces it to learn grammar, facts and some reasoning.

3,320	1M	~197k	~5.2 h
optimizer steps	tokens per step	tokens/second	wall-clock



Cross-entropy on the training batches (10-step average). The early cliff is the model learning common words and grammar; the long tail is slower knowledge gain. The extra dip after ~1,160 steps is the warmdown: lowering the learning rate lets the model settle into a better minimum.

Data: 3.48B tokens of ClimbMix

NVIDIA's ClimbMix is web text clustered by topic and re-weighted toward higher-quality clusters. Using it instead of plain FineWeb was one of the biggest wins in nanochat's speedrun history.

Data:param ratio 8 means ~8 training tokens per parameter of the scaling count. Chinchilla-style "compute-optimal" training uses ~10–20; 8 trades a little quality for a shorter run.

Optimizer: Muon + AdamW

Muon updates the big weight matrices. It orthogonalizes each update so every direction of the matrix learns at a similar rate, which trains faster than AdamW at this scale.

AdamW handles the embeddings, output layer and small scalars, where Muon does not apply.

FP8

Matrix multiplies in 121 of 132 linear layers ran in 8-bit floating point (H100 tensor cores do this at about 2× the bf16 rate). Master weights stay in fp32, so precision loss in the optimizer is avoided.

Run engineering

One script ran every stage on a rented H100. Checkpoints (model + optimizer state) were uploaded to a private Hugging Face repo every 500 steps so training could resume after a host failure, and the instance shuts itself down at the end or at a hard time limit.

4 · Chat fine-tuning (SFT)

A pretrained model only continues text. **Supervised fine-tuning** teaches it the shape of a conversation: whose turn it is, when to stop, how an assistant answers.

WHAT THE MODEL ACTUALLY SEES

`◁bos▷` `◁user_start▷` `Why is the sky blue?` `◁user_end▷`
`◁assistant_start▷`
`Sunlight scatters off air molecules, and blue light scatters most…`
`◁assistant_end▷`

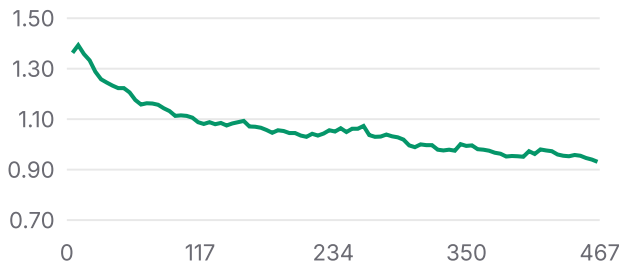
Special tokens mark turns. The loss is computed only on the **assistant's tokens** (green), so the model learns to answer, not to imitate the user.

Data mix (~790k conversations, one pass)

SOURCE	TEACHES
SmolTalk (460k)	General conversation
MMLU auxiliary train ×3	Answering multiple-choice questions
GSM8K train ×4	Step-by-step math and calculator tool use

Result

467 steps in 47 minutes; validation loss 0.296 bits/byte. Run twice from the same checkpoint, both runs reached exactly 0.2961, confirming the stage is deterministic and reproducible.



5 · Reinforcement learning on math

SFT copies answers from examples. **Reinforcement learning** instead lets the model try, scores the result, and pushes up whatever worked. Here the task is GSM8K grade-school math, where correctness is easy to check automatically.

Take 16 math problems

↓

Sample **16 answers** to each (temperature 1.0)

↓

Reward = **1 if the final number is correct**, else 0

↓

Advantage = reward − average reward for that problem
(better-than-average answers get reinforced, worse ones suppressed)

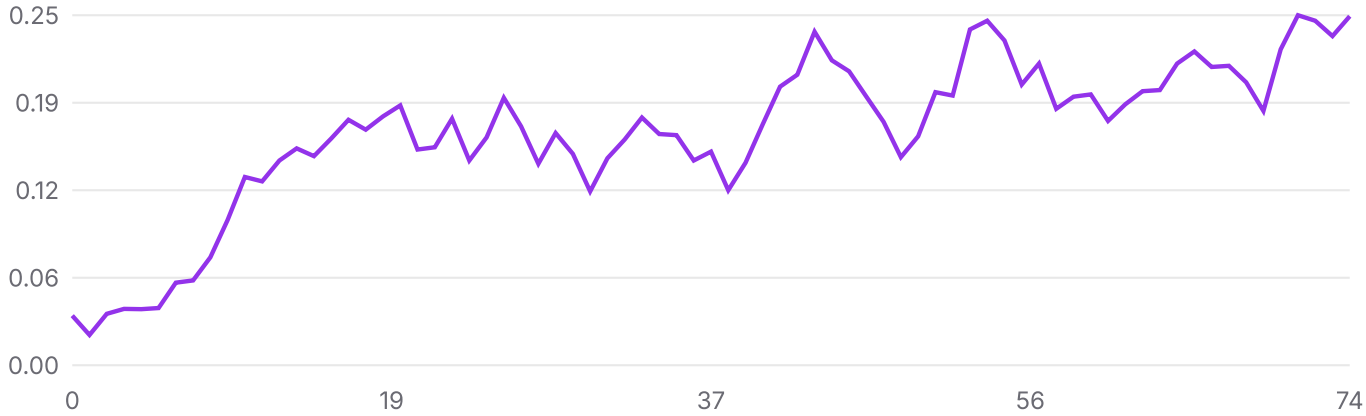
↓

Gradient step on the log-probabilities of the sampled tokens, weighted by advantage

nanochat calls this "GRPO", but it simplifies to REINFORCE with a group-mean baseline: no reference model or KL penalty, no PPO clipping, token-level normalization.

Result

1.5%	10%	75	42 min
GSM8K after SFT	GSM8K after RL	RL steps	on 1× H100



Average reward per step (fraction of sampled answers that were correct, 5-step average) rose from 4% to 25%, while answers got shorter (260 → 169 tokens on average). On held-out problems GSM8K went from 1.5% to 10.0%; ARC-Easy 55.2→55.4%, ARC-Challenge 42.9→43.9%, MMLU 35.2→35.2%, so nothing else regressed. The reference nanochat d20 reached 7.6% with RL.

6 · Benchmarks

All scores below use nanochat's own evaluation code, so they are directly comparable with each other and with the reference runs.

CORE: the base model vs GPT-2 (same harness)



CORE averages 22 tasks (HellaSwag, ARC, PIQA, LAMBADA, Winograd, SQuAD, BoolQ...), each rescaled so random guessing = 0 and perfect = 1. 500 examples per task. GPT-2 1.5B is OpenAI's published value.

Chat benchmarks

BENCHMARK	THIS MODEL	D20 REFERENCE	RANDOM
ARC-Easy	**55.2%**	38.8%	25%
ARC-Challenge	**42.9%**	—	25%
MMLU	**35.2%**	31.5%	25%
HumanEval (pass@1)	**11%**	8.5%	0%
GSM8K (first 400)	**1.5%**	4.6%	0%
GSM8K after RL (first 400)	**10%**	7.6%	0%

What each benchmark asks

ARC-Easy / Challenge · grade-school science, 4 options.
"Which form of energy does a plant use to make food? (A) light (B) sound …" Random = 25%.

MMLU · 57 subjects from high-school to professional level, 4 options. Random = 25%.

GSM8K · word problems needing several arithmetic steps; the final number must be exact. Random = 0%.

HumanEval · write a Python function from its docstring; graded by running unit tests (pass@1 = first attempt passes).

7 · Architecture experiment: MHA vs GQA vs MLA

The biggest serving cost today is the **KV cache**: every token of every conversation stores keys and values in every layer. It limits how many users fit on one GPU and how long contexts can be. I implemented **multi-head latent attention (MLA)**, DeepSeek-V2's design, in nanochat (which does not ship it) and compared it with standard multi-head attention (MHA) and grouped-query attention (GQA) under an identical recipe.

MHA · 6 KEY/VALUE HEADS

Every query head has its own keys and values. Each token stores 6 keys + 6 values of 128 numbers per layer.

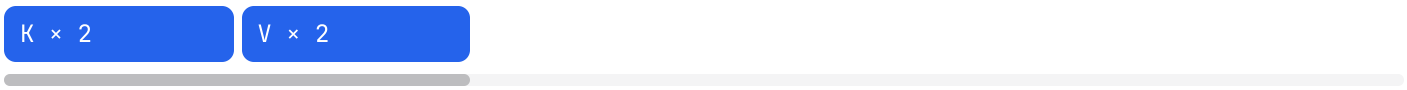
Cached per token per layer: **1,536 numbers** (1× smaller than MHA)



GQA · 2 KEY/VALUE HEADS

Three query heads share each key/value head (used by Llama 3, Qwen, Mistral). Each token stores 2 keys + 2 values.

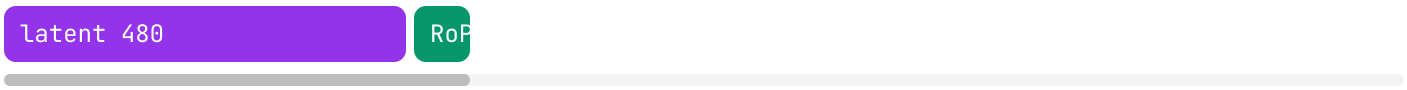
Cached per token per layer: **512 numbers** (3× smaller than MHA)



MLA · 480 LATENT + 32 ROPE

Each token stores one compressed latent (480 numbers) plus a 32-number position key shared by all heads. Keys and values for all 6 heads are reconstructed from the latent by learned up-projections.

Cached per token per layer: **512 numbers** (3× smaller than MHA)



VARIANT	KV / SEQ	SEQS IN 10 GB	VAL BITS/BYTE ↓	CORE	DECODE TOK/S (B1 / B32)
MHA	72 MB	142	0.9144	0.128	180 / 5,699
GQA (2 kv heads)	24 MB	426	0.9189	0.106	178 / 5,661
MLA (480 + 32)	24 MB	426	0.9209	0.099	89 / 2,603

Depth 12 (768 wide, 6 heads), 0.52B tokens each, same data, seed, optimizer and schedule; value embeddings off in all three. KV sizes for one 2,048-token sequence in bf16. Decode on one H100 with a 1,024-token prompt.

1 · A 3× smaller cache costs ~0.5–0.7% loss. GQA and MLA both fit 426 sequences in a 10 GB cache vs 142 for MHA, for a small quality hit. This is the trade-off production models make.

2 · At equal memory, MLA did not beat GQA here. The gap (0.002 bits/byte) is within single-seed noise. DeepSeek reports MLA ahead at much larger scale; no MLA-specific hyperparameters were tuned.

3 · MLA decoded ~2× slower. nanochat normalizes full keys (QK-norm), which blocks MLA's weight-absorption trick, so the cached latents are re-expanded into keys and values at every step.

Limits. One seed per variant, small model, short run. CORE at this size is noisy, so validation bits/byte is the main metric.

8 · Inference & serving

Generating text is a loop: run the model, pick the next token, append it, repeat. The **KV cache** stores each token's attention keys and values, so every new token only needs one new row of computation instead of re-reading the whole sequence.

Generate **200** tokens after a 300-token prompt

Without KV cache

80,100 positions

With KV cache

500 positions (160× less)



Token positions processed in total. Without a cache the work grows with the square of the length.

WHERE IT RUNS	HARDWARE	SPEED
Hugging Face Space (public demo)	free 2-vCPU CPU	~7 tok/s
Apple Silicon Mac	CPU, 2 threads	~16–24 tok/s
On-demand GPU	RTX A4000 class	~53 tok/s

One `app.py` serves a Gradio chat UI and an **OpenAI-compatible API** (`/v1/chat/completions` with streaming). A small control plane launches the GPU on demand, relays requests through a fixed HTTPS address, and destroys the instance after 45 idle minutes.

Tried and rejected: int8 quantization. On CPU it was slower (13.7 vs 16.2 tok/s) and lost ~6 points on ARC-Easy (48.0% vs 54.2%), so the demo stays in fp32.

9 · Glossary

Token	A piece of text (word, sub-word or symbol) mapped to an integer id. Models read and write tokens.
BPE tokenizer	Byte-pair encoding: builds a vocabulary by repeatedly merging the most frequent adjacent byte pairs. Ours has 32,768 entries.
Parameters	The learned numbers in the model (weights). 897M total; 393M in the transformer matrices that do the compute.
Embedding	A learned vector per token id (a lookup table). Converts ids into the 1280-number vectors the transformer processes.
Value embedding	nanochat addition: a second per-token lookup table added into attention values on alternate layers, through a learned gate.
Transformer	Neural network architecture built from stacked attention + MLP blocks with residual connections. Basis of GPT, Llama, Claude.
Decoder-only	Transformer that only predicts the next token from previous ones (causal), as opposed to encoder-decoder models.
Self-attention	Mechanism that lets each token gather information from earlier tokens, weighted by learned relevance (queries vs keys).
Attention head	One independent attention computation. 10 heads × 128 dimensions = 1280.

GQA	Grouped-query attention: several query heads share key/value heads to shrink the KV cache. Supported, but here n_kv_head = n_head (10), so it is plain multi-head attention.
RoPE	Rotary position embeddings: encode token position by rotating query/key vectors, so attention depends on relative distance.
QK-norm	Normalizing queries and keys before attention. Keeps attention scores in a sane range and stabilizes training.
RMSNorm	Rescales each vector to unit root-mean-square. nanochat uses it without a learned scale.
relu²	Activation function $\max(0, x)^2$. Simple and works well in recent speedruns.
MLP	The per-token feed-forward part of a block: 1280 → 5120 → 1280.
Residual connection	Adding a block's input back to its output, so layers learn corrections. Essential for deep networks.
Sliding-window attention	Each token only attends to the last N tokens (768 here) instead of all previous ones. Cheaper; used on 15 of 20 layers.
FlashAttention-3	Fast, memory-efficient attention kernel for H100 GPUs.
Logit soft-cap	Squashing output scores with $15 \cdot \tanh(x/15)$ so no logit exceeds ±15. Prevents instability.
Context length	Maximum tokens the model sees at once: 2,048.
Pretraining	Training on raw text to predict the next token. Where knowledge and language ability come from.
Loss / cross-entropy	How surprised the model is by the true next token, averaged. Lower is better.
Bits per byte (bpb)	Loss converted to bits per byte of raw text. Tokenizer-independent, so models with different vocabularies can be compared.
Validation set	Held-out data never trained on, used to check the model generalizes.
Step / batch	One optimizer update. Each step here used a batch of ~1M tokens (2048-token sequences, gradient accumulation).
Gradient accumulation	Summing gradients over several small forward/backward passes to simulate a larger batch than fits in GPU memory.

Learning rate & warmdown	Step size of updates. Warmup ramps it up at the start; warmdown decays it linearly over the last 65% of training.
Muon	Optimizer for weight matrices that orthogonalizes updates (Newton–Schulz iterations). Faster convergence than AdamW at this scale.
AdamW	Standard adaptive optimizer with decoupled weight decay; used here for embeddings, the output layer and scalars.
FP8 / bf16 / fp32	Number formats of 8, 16 and 32 bits. Lower precision is faster; master weights are kept in fp32 for stable updates.
MFU	Model FLOPs utilization: fraction of the GPU's peak math throughput actually used.
Data:param ratio	Training tokens per parameter. Chinchilla scaling suggests ~20 is compute-optimal; this run used 8.
Chinchilla scaling	DeepMind's 2022 finding on how to split a compute budget between model size and data for the lowest loss.
ClimbMix	NVIDIA's curated pretraining corpus: web text clustered by topic and re-weighted toward high-quality clusters.
SFT	Supervised fine-tuning on example conversations; teaches chat format and answer style. Loss only on assistant tokens.
SmolTalk	Hugging Face's open dataset of synthetic assistant conversations, used for SFT.
RL / RLHF	Reinforcement learning: the model generates, a reward scores the output, and good outputs are reinforced. RLHF uses human-preference rewards; here the reward is answer correctness.
GRPO / REINFORCE	Policy-gradient methods. GRPO compares several samples for the same prompt; nanochat's version is REINFORCE with a group-mean baseline.
Advantage	How much better a sample did than the group average; the weight applied to its gradient in RL.
CORE	DCLM benchmark suite: 22 tasks averaged after rescaling so random = 0. Used to compare base (pretrained) models.
Centered accuracy	Accuracy rescaled so random guessing scores 0 and perfect scores 1.
ARC / MMLU / GSM8K / HumanEval	Science QA / 57-subject knowledge / grade-school math / Python coding benchmarks.

pass@1 / pass@k	Probability that the first answer (or any of k sampled answers) is correct.
Few-shot / zero-shot	Whether the prompt includes worked examples before the question (few-shot) or not (zero-shot).
Temperature	Sampling randomness. 0 = always pick the most likely token; higher = more varied.
KV cache	Stored attention keys/values of previous tokens, so generation does not recompute them for every new token.
MLA	Multi-head latent attention (DeepSeek-V2): caches one compressed latent per token and reconstructs every head's keys/values from it, plus a small shared RoPE key. Implemented here for nanochat.
Weight absorption	MLA inference trick: fold the key up-projection into the query so attention runs directly on the cached latent. Blocked here because nanochat normalizes full keys (QK-norm).
Ablation	An experiment that changes one component while holding everything else fixed, to measure that component's effect.
Quantization (int8)	Storing weights in 8-bit integers to save memory/compute. Tested here and rejected: slower on CPU and less accurate.
Checkpoint	Saved model (and optimizer) state; lets training resume or the model be served.
OpenAI-compatible API	Server speaking the same /v1/chat/completions protocol as OpenAI, so standard clients work unchanged.
Hugging Face Space	Free hosting for ML demos on Hugging Face; ours runs the chat UI on a 2-vCPU CPU.

10 · Likely interview questions

▼ **Why train from scratch instead of fine-tuning an open model?**

To own and understand every stage: tokenizer, pretraining, SFT, RL, evaluation and serving. Fine-tuning hides the hardest parts. The from-scratch constraint also makes the benchmark comparisons meaningful.

▼ What did you build versus what nanochat provided?

nanochat provides the model, optimizer and training/eval scripts. I chose the configuration (depth 20, data:param 8, FP8), sized the run with a throughput probe, wrote the unattended training/checkpointing/teardown pipeline, ran the evaluations (including scoring GPT-2 with the same harness), fitted the RL run to a fixed compute budget by capping steps while keeping the learning-rate decay intact, ran the int8 experiment, and built the serving stack (Gradio + OpenAI API, CPU Space, on-demand GPU control plane).

▼ Why is it 897M parameters but you compare it to 355M GPT-2?

503M of the 897M are embedding lookup tables (including value embeddings), which cost memory but almost no compute. The 393M transformer weights are what do the work per token, so they are the fairer comparison point. On CORE it beats GPT-2 medium (0.236 vs 0.192) and approaches GPT-2 XL (0.257).

▼ What is bits per byte and why report it?

Loss per byte of raw text instead of per token. Token-level loss depends on the tokenizer (bigger tokens make each prediction harder), so bpb lets models with different vocabularies be compared fairly.

▼ How did you decide model size and training length?

A short probe on the target GPU measured real throughput (~197k tokens/s for d20). From that I picked depth 20 with a data:param ratio of 8, slightly below compute-optimal, which fit the time budget while staying near the efficient frontier.

▼ Why was GSM8K so low, and what did RL change?

Multi-step arithmetic needs reliable reasoning, which small models lack: after SFT it solved 1.5%. 75 steps of RL (42 min on one H100) rewarded correct final answers and raised it to 10.0% on the same 400 problems, above the reference nanochat d20 (7.6%), with no regression on ARC or MMLU. Answers also got shorter, as the model stopped rambling.

▼ Why did the benchmarks beat the original nanochat d20 with fewer tokens?

Same depth, newer nanochat code: better data (ClimbMix), architectural tweaks (value embeddings, smear/backout, sliding windows) and tuned optimization. Same harness, so the comparison is fair.

▼ How did you make the training run reliable?

Checkpoints with optimizer state uploaded every 500 steps (resume after host failure), hard time caps with automatic teardown, secrets kept out of logs, and the throughput/stage probe before committing to the full run. The chat fine-tune was later re-run from the saved checkpoint and reproduced the same validation loss exactly.

▼ What did you add on top of nanochat besides running it?

An architecture experiment nanochat does not ship: I implemented multi-head latent attention (latent KV cache, decoupled RoPE key, compact cache in the generation engine, tests that cached decoding matches the full forward pass) and ran a controlled MHA vs GQA vs MLA ablation at equal KV-cache budget, measuring quality, memory per sequence and decode throughput. Plus the same-harness GPT-2 baseline, the budget-capped RL run, the int8 study and the serving stack.

▼ Why did MLA not beat GQA, when DeepSeek says it should?

At depth 12 on 0.5B tokens the two were within single-seed noise (0.9189 vs 0.9209 bits/byte). DeepSeek's results are at far larger scale, use tuned latent sizes and query compression, and their keys are not re-normalized after up-projection. I would run multiple seeds and sweep the latent width before drawing a stronger conclusion.

▼ Why was MLA slower at generation?

Normally MLA folds the key up-projection into the query (weight absorption) and attends directly over the cached latent. nanochat applies QK-norm to the full keys, a nonlinearity between the up-projection and the dot product, so absorption is impossible and every step re-expands all cached latents into keys and values: 2.2x slower at batch 32. Fixing it means moving the norm onto the latent or dropping QK-norm, a stability trade-off.

▼ What would you do with 10× more compute?

Train deeper (d26 reaches GPT-2 XL quality), use a compute-optimal data ratio, extend RL beyond GSM8K, and add a preference-tuning stage for chat quality.

▼ How is it served?

One app serves a Gradio chat UI and an OpenAI-compatible streaming API. A free CPU Space hosts the public demo (~7 tok/s); for speed, a control plane launches a small GPU on demand behind a fixed HTTPS relay (~53 tok/s) and destroys it after 45 idle minutes.

▼ What did not work?

int8 dynamic quantization for the CPU demo: slower (13.7 vs 16.2 tok/s) and ~6 points worse on ARC-Easy, so it was dropped. Also, a checkpoint uploader skipped files younger than 60 s and missed the final SFT checkpoint; it was fixed with explicit post-training upload verification.

▼ What are the model's limitations?

It hallucinates facts confidently, is weak at arithmetic and long arguments (it confuses who said what), has a 2,048-token context, is English-only, and has no dedicated safety tuning.

Numbers on this page come from the run's logs and nanochat's evaluation code (commit 92d63d4). Source: github.com/rsumit123/private-llm.